Open in Colab

Strings

A *string* in Python is just a sequence of (unicode) characters.

To create a string, write the characters surrounding them by either `'` or `"` marks.

- If the string is started by `'` character then it must be ended by the same `'` character.
- If the string is started by `"` character then it must be ended by the same `"` character.

In [1]:
```
'hello'
```

Out[1]:
```
'hello'
```

We will use builtin `print()` function to write to the console.

In [2]:
```
print('hello')
```

```
hello
```

In [3]:
```
print("hello")
```

```
hello
```

Special Characters In Strings

If you want to write `'` character inside a string then you can do this by ethier

- escaping the `'` character by `\'` or
- using `"` character to surround the string.

If you want to write `\` itself inside a string then use `\\`.

In [4]:
```
print("I'm learning to write \', \" and \\ characters inside a string.")
```

```
I'm learning to write ', " and \ characters inside a string.
```

Similarly to write `"` character inside a string surrounded by `"` character then you can do this by ethier

- escaping the `"` character by `\"` or
- using `'` character.

Multiline Strings & White Spaces

Ordinary strings must start and end in one line. (You can not break the line when writing a string between `'` or `"` characters.)

If you want to create a sting with a new line character you can use: `\n`

In [5]:
```
print('hello\nworld!')
```

```
hello
world!
```

If you want to write tab character use `\t`. Tab character puts 1 to 8 spaces to align the text on

the console.

In [6]:
```
print('hello\tworld!')
```

```
hello   world!
```

`\r` is the cariage return character.

In [7]:
```
print('hello\rworld!')
```

```
world!
```

Another way of creating multiline strings in Python is to use `'''` or `"""` to start and end the string. (Exactly three times)

In [8]:
```
print('''Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.''')
```

```
Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.
```

In [9]:
```
print("""Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.""")
```

```
Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.
```

Another way of creating long strings is to write multiple strings with only white spaces between them. This is useful when using different syntax for different parts of the string:

In [10]:
```
print("He's " 'the so-called "champion"')
```

```
He's the so-called "champion"
```

In fact, you can write multiple strings in a single `print()` call by seperating them with `,`.

`print()` function will replace them with a space.

In [11]:
```
print("He's", 'the so-called "champion"')
```

```
He's the so-called "champion"
```

You can change this behaviour by passing optional `sep=` parameter to the `print()` function

In [12]:
```
print("Hello", "world", sep="-")
```

```
Hello-world
```

By default `print()` function adds a `\n` character at the end of its output. You can change this behaviour passing optional `end=` parameter to the `print()` function.

In [13]:
```
print("Hello", end=" ")
print("World")
```

```
Hello World
```

Numbers As Strings

The integer `12` and the string `"12"` are different. You can see this by:

In [14]:
```
12 == '12'
```

Out[14]:
```
False
```

To convert a number to a string use `str()` function.

In [15]:
```
str(12)
```

Out[15]:
```
'12'
```

To convert a string to integer use `int()` function.

In [16]:
```
int('12')
```

Out[16]:
```
12
```

Similarly you can use `float()` function to convert a string to floating point number and `complex()` to convert to a complex number.

In [17]:
```
complex('12')
```

Out[17]:
```
(12+0j)
```

Getting A String From The User

`input(prompt)` function can be used to get a string from the user with an optional `prompt` string.

In [18]:
```
print("Hello", input("What is your name? "))
```

```
Hello
```

Warning: When evaluating an expression, Python first evaluates the expressions between paranthesis. Therefore in the previous example first `input("Enter your name: ")` is evaluated. Then its return value is used to evaluate `print(...)`.

Note that `input()` function always returns a string. If you want a number then you should convert it with `int()`, `float()` or `complex()` functions.

In [20]:
```
print("You born after:", 2020 - int(input("How old are you? ")))
```

```
You born after: 2000
```

This conversion is needed to do integer arithmetic on input. Otherwise Python will rise an error.

Operations on Strings:

You can get the length of a string by the builtin `len()` function.

In [ ]:
```
len('hello world!')
```

Strings can be concatenated by `+` operator.

In [ ]:
```
print('hello'+' '+'world'+'!')
```

Also you can use `*` to repeat the same string multiple times.

In [ ]:
```
print('hello world!'+'!'*3)
```

In [ ]:
```
print(('hello world!'+'!\n') * 3)
```

To convert a string to uppercase letters use `.upper()` method.

In [ ]:
```
print('hello world!'.upper())
```

To convert a string to lowercase letters use `.lower()` method.

In [ ]:
```
print('hello world!'.lower())
```

To convert the first letter of a string use `.capitalize()` method.

In [ ]:
```
print('hello world!'.capitalize())
```

To center a string use `.center(n)` method where `n` is the width of the line.

In [ ]:
```
print('hello world!'.center(30))
```

Comparing Strings

You can compare two strings with `<`, `<=`, `>`, `>=`, `==` and `!=` operations.

In [ ]:
```
"hello world!" == 'hello world!'
```

In [ ]:
```
'hello world!' > 'hello world!'
```

Here the comparision is done in *lexicographic* order. i.e.

- Initially, Python compares the first characters of both strings.
- If the first characters of both strings are same then the comparission is done between the second characters.
- If the second characters are also same then the third.
- And so on.

Substrings

To test if a string is contained in another string use `in` keyword.

In [ ]:
```
'world' in 'hello world!'
```

To test if a string is **not** contained in another string use `not in` keyword.

In [ ]:
```
'hello' not in 'hello world!'
```

To count the number of substring in a string use `.count()` method.

In [ ]:
```
'hello world!'.count('l')
```

To find how many characters are there in the string before a given string use `.index()` method.

In [ ]:
```
'hello world!'.index("world")
```

Accessing String Characters

To access:

- the first character of the string use `[0]`,
- the second character of the string use `[1]`,
- and so on.

This is the number of characters to be skipped to access that character.

It is called the *index* of that character and can be found by `.index()` method.

In [ ]:
```
'hello world!'[0]
```

In [ ]:
```
'hello world!'.index('h')
```

To access the last character of a string use `[-1]`. You can use negative numbers to index the characters from end.

In [ ]:
```
'hello world!'[-1]
```

Warning: To index a string character from the beginning count like `0,1,2,...` To index a string character from the end count like `-1,-2,-3,...`

Accessing Substrings

To get a substring use `[start:end]` notation. This will get all the characters starting with (including) `start` up to (but not including) `end`.

In [ ]:
```
'hello world!'[1:4]
```

You can also use negative indexes here.

In [ ]:
```
'hello world!'[0:-1]
```

If you ommit the `start` index then Python assumes it `0`. So to get the first `3` characters:

In [ ]:
```
'hello world!'[:3]
```

If you ommit the `end` index then Python assumes it the length of the string. So to get the last `3` characters:

In [ ]:
```
'hello world!'[-3:]
```

If your indexes are out of valid ranges then Python assumes them like empty:

In [ ]:
```
'hello world!'[-100:100]
```

You can skip some characters with `[start:end:skip]` notation. For example to get only the characters in even positions:

In [ ]:
```
print('hello world!'[::2])
```

To print the string but in reverse order:

In [ ]:
```
print('hello world!'[::-1])
```

f-Strings

Strings that start with `f` are called *f-string*. Inside an f-string between braces `{}` you can write any Python expression. Python will run the expression and replace it with the result.

In [ ]:
```
print(f'12+7={12+7}')
```

Modifers

You can use f-strings with modifiers (`{:}`) to change the representation of objects in f-strings

Here are some examples:

In [ ]:
```
print(f'0.1*3={.1*3:.2f}')
```

Here the modifier `.2f` instructs the Python to use only the first 2 digits after `.` to show a floating point number.

In [ ]:
```
print(f'Hexadecimal represenation of 125 is 0x{125:X}')
```

In [ ]:
```
print(f'Binary expansion of 125 is 0b{125:b}')
```

In [ ]:
```
print(f'Scientific form of 125 is {125:e}')
```